HANGZHOU DIANZI UNIVERSITY

Information Security Laboratory

LABORATORY WORK No. 1

Control Questions

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant	N/A (not assigned)
Student	李泽涛 / LiZetao
HDU number	23320332
Group	23320113
Submission date	19 October 2026

Answers to the eight control questions

The answers below refer to the implementation, the CI/CD pipeline and the CVE case study delivered in the main report (Lab1_Report.docx). The main report itself does not duplicate these answers.

Q1. Why is password hashing with algorithms like bcrypt preferable to using SHA-256?

SHA-256 is designed to be fast. A modern GPU can compute billions of SHA-256 hashes per second, so if a password database leaks, an attacker can brute-force short or common passwords, and precomputed rainbow tables defeat unsalted digests almost instantly. bcrypt is deliberately expensive: it is based on the Blowfish key schedule, uses a per-password random salt, and has a tunable cost factor (work factor). Raising the cost by one doubles the work for both the defender and the attacker; a cost of 12 means thousands of iterations, so guessing is orders of magnitude slower. The salt makes rainbow tables useless and ensures two users with the same password get different hashes. bcrypt also has a small memory footprint compared with SHA-256, which is not a problem for a login endpoint but is a limitation; Argon2id is the modern first choice because it is memory-hard and resists GPU/ASIC cracking even better. SHA-256 remains appropriate for integrity checks and HMAC, not for storing passwords. In this project passwords are hashed with bcrypt (cost 12, unique salt) and verified with bcrypt.checkpw, never with a plain SHA-256 digest.

Q2. What is the main difference between the SAST and DAST approaches to security testing?

SAST (Static Application Security Testing) is white-box testing: it analyses source code, bytecode or binaries without executing the application. It can run in the IDE and in CI on every commit, finds dangerous patterns early (for example string-concatenated SQL, eval, weak crypto) and can point to the exact file and line. Its weaknesses are false positives, inability to see runtime configuration, framework magic, authentication state or third-party service behaviour, and it cannot prove exploitability.

DAST (Dynamic Application Security Testing) is black-box testing: it runs the deployed application and sends requests or drives the UI, then inspects responses. It sees the application as an attacker does, so it can find runtime problems such as missing authentication on an endpoint, misconfiguration, session issues and real injection behaviour, with fewer false positives about reachability. Its weaknesses are late feedback (needs a running environment), incomplete coverage of code paths, and false negatives when a payload is not triggered.

The two are complementary: SAST shifts detection left and covers code; DAST validates what is actually exposed. In this work Bandit is the SAST tool in CI and OWASP ZAP (Work 2) is the DAST tool; a mature pipeline runs both plus SCA.

Q3. Describe the mechanism of a JWT token. What information does it contain and how does the server verify its authenticity?

A JWT is three Base64URL-encoded parts separated by dots: header.payload.signature. The header declares the signing algorithm and type ({"alg":"HS256","typ":"JWT"}). The payload is a set of claims: registered claims such as sub (subject/user id), iat (issued at), nbf (not before), exp (expiry), iss (issuer), aud (audience) and jti (unique token id), plus application claims such as a username. The signature is computed over base64url(header) + "." + base64url(payload) using the server's secret (HMAC-SHA256) or a private key (RS256/ES256).

The server does not "decode and trust" the token. It recomputes the signature with its key and compares it in constant time, then verifies that alg is the expected algorithm (never accepts none or an attacker-chosen algorithm), checks exp/nbf, and validates iss and aud. Only then does it use sub to load the user. Because the payload is only signed, not encrypted, it must never contain passwords, secrets or personal data. Tokens are stateless, which scales well but makes immediate revocation hard; short TTLs, a jti denylist or refresh-token

rotation are used instead. In this project the token is HS256, valid for 15 minutes, and jwt.decode is called with algorithms=["HS256"], audience and issuer, so alg=none, tampered and expired tokens are all rejected.

Q4. What risks arise if you do not audit third-party libraries (dependencies) in a project?

Modern applications are mostly third-party code: this small Flask API pulls in dozens of transitive packages, and the two-package Node demo installed 77 packages. Without auditing, an application inherits every known vulnerability in that tree. A single outdated template engine can turn into remote code execution — exactly the case of CVE-2022-29078 in EJS 3.1.6, exploited in this work. Other risks include supply-chain attacks (typosquatting, compromised maintainer accounts, malicious install scripts), abandoned packages with no security response, incompatible or unexpectedly upgraded transitive versions, licence violations, and vulnerable build/test tooling that leaks into CI. Because dependencies change constantly, a version that was clean at release can become vulnerable later; without continuous SCA and an SBOM you do not know what you ship. The consequences range from data breaches and RCE to failed compliance audits and emergency patching under pressure. The mitigation is to pin versions, run SCA on every build (pip-audit, OWASP Dependency-Check, npm audit), track advisories, remove unused packages, and keep an SBOM so affected systems can be found quickly when the next Log4Shell-style issue appears.

Q5. What advantages does using Docker and Vulhub provide for studying cybersecurity compared to installing vulnerable software directly on your machine?

Docker isolates the vulnerable service in a container with its own filesystem, network namespace and pinned dependencies. The host stays clean, and a mistake (or a deliberately malicious payload) is confined to a disposable container that can be deleted with docker compose down -v. The environment is reproducible: the same image and compose file give the same versions on every student's laptop and on CI, which matters when a CVE only affects a specific version. Dependency conflicts disappear because the vulnerable old library lives inside the image, not in the system Python/Node installation. Multi- service targets (database, web server, application) start with one command, snapshots allow returning to a clean state, and ports can be published only on 127.0.0.1.

Vulhub adds ready-made compose files and READMEs with the exact vulnerable version, launch command and exploitation steps, so the learner focuses on the vulnerability rather than on reconstructing an old environment. Installing vulnerable software directly risks infecting the host, breaking system packages, leaving backdoors and being unable to cleanly uninstall. Docker is not perfect isolation (shared kernel, privileged daemon, image supply chain), so containers should still be treated as untrusted and never exposed to the internet.

Q6. Describe the difference between a vulnerability and an exploit. What were you reproducing in this work?

A vulnerability is a weakness in code, configuration or design that can lead to a security impact — for example EJS 3.1.6 concatenating the outputFunctionName option into generated JavaScript without validation, combined with an application that forwards req.query into res.render. The vulnerability exists whether or not anybody attacks it. An exploit is the concrete technique, input or tool that takes advantage of the vulnerability to produce an effect: here, a crafted HTTP query parameter that closes the generated var statement, calls process.mainModule.require("child_process").execSync(...), and keeps the generated code syntactically valid with a trailing s.

In this work I reproduced the exploitation of CVE-2022-29078 against a controlled EJS 3.1.6 environment: the request made the server execute whoami and write the output to pwned.txt, while the HTTP response still returned the normal page with status 200. The vulnerability is the unsafe option injection; the exploit is the specific request and payload. Reproducing the exploit proves the vulnerability is reachable and shows its

impact; the root-cause analysis then explains why the payload works, and the fix removes the condition that makes it possible.

Q7. Why, after applying a fix (patch), is it critically important to check that the main functionality of the application has not been damaged (regression testing)?

A security patch changes code, configuration or a dependency version, and any change can break behaviour that users depend on. A version bump may alter APIs, error handling, performance or template semantics; a code fix may reject legitimate input, change response formats or break authentication flows. Without regression testing you can ship a "secure" application that no longer logs users in, loses data or returns 500 on normal requests — the cure becomes worse than the disease. Regression tests also catch incomplete fixes: if the original attack still works through another path, the vulnerability was not really removed. They document the expected behaviour so future changes cannot silently reintroduce the flaw, and they give confidence to release quickly under pressure.

In this work the fixed application was re-tested in three ways: the exploit request no longer created pwned.txt; the normal page still rendered Hello, guest!; and the library rejected the malicious option with outputFunctionName is not a valid JS identifier. In parallel, the API's 21 automated tests (login, token validation, data access, escaping, rate limiting) all pass after the security changes, which is exactly the regression check the manual asks for.

Q8. How does knowledge and the ability to reproduce known vulnerabilities (CVEs) help in the work of a security specialist (for example, a pentester or SOC analyst)?

For a pentester, CVE reproduction builds a library of real attack primitives: what a payload looks like, which preconditions are required, how the application responds, and how to verify impact without damaging the target. It makes penetration tests faster and more realistic, helps prioritise findings by exploitability rather than by scanner score, and allows the tester to prove a finding with a safe proof of concept and then verify that the remediation actually blocks it.

For a SOC analyst, the same knowledge improves detection and response. Reproducing a CVE shows which log entries, HTTP parameters, process creations or file writes the exploit produces, so detection engineers can write SIEM rules, WAF signatures and IDS patterns that catch the behaviour rather than a single IP. It helps triage alerts (is this the known EJS payload or a false positive?), supports threat hunting and incident response, and helps vulnerability management prioritise internet-facing systems running an affected version, especially when the CVE is in CISA KEV or has a public exploit.

It also develops safe habits: working in isolated labs, documenting exact versions, using minimal payloads, and cleaning up afterwards. That combination — offensive understanding plus defensive mapping — is what turns a CVE identifier into actionable security work.